

ΜΥΕ023 – ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ ΚΑΙ
ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ 2023-24

OPENMP
ΣΕΤ ΑΣΚΗΣΕΩΝ #1

Περιεχόμενα

Εισαγωγή.....	2
Άσκηση 1: Παραλληλοποίηση εύρεσης πρώτων αριθμών.....	3
Το πρόβλημα.....	3
Ανάλυση Κώδικα.....	3
Γραφικές Παραστάσεις - Χρονομετρήσεις.....	7
Συμπεράσματα	9
Άσκηση 2: Θόλωση εικόνων	10
Το πρόβλημα.....	10
Ανάλυση Κώδικα omp_loops.....	10
Γραφικές Παραστάσεις - Χρονομετρήσεις.....	11
Συμπεράσματα	15
Ανάλυση Κώδικα omp_tasks	15
Γραφικές Παραστάσεις - Χρονομετρήσεις.....	16
Συμπεράσματα	17
Άσκηση 3: Task dependencies	18

Εισαγωγή

Η 1^η σειρά ασκήσεων εστιάζει γενικά στην εκμάθηση και εφαρμογή του OpenMP, ενός πρότυπου για παράλληλο προγραμματισμό, με στόχο να βελτιωθεί η απόδοση επεξεργασίας μέσω της παράλληλης εκτέλεσης κώδικα σε πολυπύρρηνα συστήματα. Κάθε άσκηση εξετάζει διαφορετικές πτυχές της παραλληλοποίησης:

1. Παραλληλοποίηση υπολογιστικά απαιτητικών εργασιών: Όπως η εύρεση πρώτων αριθμών, όπου η εργασία μπορεί να διαμοιραστεί σε πολλαπλά νήματα για να μειωθεί ο συνολικός χρόνος εκτέλεσης.
2. Παραλληλοποίηση επεξεργασίας δεδομένων: Στην εφαρμογή Gaussian blur σε εικόνες, όπου οι υπολογισμοί για διαφορετικά τμήματα της εικόνας γίνονται παράλληλα, βελτιώνοντας την απόδοση κατά την επεξεργασία μεγάλων εικόνων.
3. Διαχείριση εξαρτήσεων μεταξύ tasks: Μια πρώτη εισαγωγή στο πώς μπορούμε να εκμεταλλευτούμε τις δυνατότητες του OpenMP για διαχείριση εξαρτήσεων μεταξύ διαφορετικών παράλληλων εργασιών, διερευνώντας (μέσω παραδείγματος) πώς η αποδοτική διαχείριση αυτών των εξαρτήσεων μπορεί να οδηγήσει σε πιο αποτελεσματική και οργανωμένη εκτέλεση παράλληλου κώδικα.

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	opti7020ws21
Επεξεργαστής	Intel i5-4590
Πλήθος πυρήνων	4
Μεταφραστής	gcc v11.2.0

Άσκηση 1: Παραλληλοποίηση εύρεσης πρώτων αριθμών

Το πρόβλημα

Μας δίνεται ένα σειριακό πρόγραμμα στο οποίο, δεδομένου του N , η συνάρτηση `serial_primes()` υπολογίζει το πλήθος των πρώτων αριθμών καθώς και τον μεγαλύτερο πρώτο αριθμό μέχρι και το N . Μας ζητείται να συμπληρώσουμε τη συνάρτηση `openmp_primes()` ώστε να κάνει τους ίδιους υπολογισμούς παράλληλα, χρησιμοποιώντας το OpenMP και να βρεθεί η βέλτιστη διαμοίραση της δουλειάς.

Ανάλυση Κώδικα

Για την παραλληλοποίηση της συνάρτησης `serial_primes()` χρησιμοποίησα βασικές εντολές της OpenMP. Παρακάτω αναφέρονται οι βασικές αλλαγές και προσθήκες που έγιναν στον αρχικό κώδικα:

- **Δήλωση Παράλληλης Περιοχής:**

Χρησιμοποίησα την οδηγία `#pragma omp parallel for` για να ορίσω μια παράλληλη επανάληψη.

- **Ιδιωτικές Μεταβλητές:**

Οι μεταβλητές `i`, `num`, `divisor`, `quotient`, και `remainder` δηλώθηκαν ως `private` μέσω της οδηγίας `private(i, num, divisor, quotient, remainder)`, εξασφαλίζοντας ότι κάθε νήμα θα έχει τις δικές του ανεξάρτητες αντιγραφές αυτών των μεταβλητών, αποφεύγοντας τη διακοπή από άλλα νήματα.

- **Παράλληλες Ενημερώσεις Μεταβλητών:**

Χρησιμοποίησα τις οδηγίες `reduction(+:count)` και `reduction(max:lastprime)` για να καταχωρήσω συνδυαστικά τις τιμές των μεταβλητών `count` και `lastprime` αντίστοιχα. Οι μεταβλητές αυτές ενημερώνονται με ασφαλή τρόπο από όλα τα νήματα, συνδυάζοντας τις τιμές από διαφορετικά νήματα στο τέλος του παράλληλου βρόχου.

- **Κοινόχρηστες Μεταβλητές:**

Η μεταβλητή n χρησιμοποιείται για να καθορίσει το εύρος των αριθμών στους οποίους θα γίνει ο έλεγχος για τους πρώτους αριθμούς. Η χρήση της ως κοινόχρηστη (shared) μεταβλητή είναι αποδοτικότερη από το να δημιουργηθεί ένα ιδιωτικό αντίγραφο για κάθε νήμα, καθώς η n απλά διαβάζεται και δεν τροποποιείται.

- **Κώδικας**

Στον παραδοτέο κώδικα κρατήσαμε μία από τις βέλτιστες περιπτώσεις.

```
50 void openmp_primes(long int n) {
51     long int i, num, divisor, quotient, remainder;
52
53     if (n < 2) return;
54     count = 1;                /* 2 is the first prime */
55     lastprime = 2;
56
57     omp_set_dynamic(0);        /* nothing to do with the dynamic schedule! */
58     omp_set_num_threads(4);    //schedule(runtime): for the different types of parallelism
59     #pragma omp parallel for schedule(dynamic, 100000) private(i, num, divisor, quotient, remainder) reduction(+:count) reduction(max:lastprime) shared(n)
60     for (i = 0; i < (n-1)/2; ++i) { /* For every odd number */
61         num = 2*i + 3;
62
63         divisor = 1;
64         do
65         {
66             divisor += 2;        /* Divide by the next odd */
67             quotient = num / divisor;
68             remainder = num % divisor;
69         } while (remainder && divisor <= quotient); /* Don't go past sqrt */
70
71         if (remainder || divisor == num) /* num is prime */
72         {
73             count++;
74             lastprime = num;
75         }
76     }
77 }
```

- **Βοηθητικός Κώδικας**

Βοηθητική συνάρτηση για χρονομέτρηση:

```
double timeit(void (*func)(long int), long int arg) {  
    struct timeval start, end;  
    gettimeofday(&start, NULL);  
    func(arg); // Pass the argument to the function  
    gettimeofday(&end, NULL);  
    return (double) (end.tv_usec - start.tv_usec) / 1000000  
        + (double) (end.tv_sec - start.tv_sec);  
}
```

Βοηθητικός κώδικας για την εύρεση μέσων τιμών για κάθε περίπτωση:

```
int numThreads[] = {2, 3, 4};

omp_sched_t schedules[] = {omp_sched_static, omp_sched_dynamic, omp_sched_guided};
char* scheduleNames[] = {"static", "dynamic", "guided"};
int chunkSizes[] = {100000, 250000, 500000, 1000000, 2000000}; // Example chunk sizes
double sumExectime = 0;

int runs = 4;

for (int threadIndex = 0; threadIndex < 3; threadIndex++) {
    for (int schedIndex = 0; schedIndex < 3; schedIndex++) {
        for (int chunkIndex = 0; chunkIndex < 5; chunkIndex++) {
            printf("Running with %d threads, schedule %s, chunk size %d four times.\n",
                numThreads[threadIndex], scheduleNames[schedIndex], chunkSizes[chunkIndex]);

            for (int run = 0; run < runs; run++){
                omp_set_dynamic(0);

                omp_set_num_threads(numThreads[threadIndex]);

                omp_set_schedule(schedules[schedIndex], chunkSizes[chunkIndex]);

                count = 0;

                lastprime = 0;

                sumExectime += timeit(openmp_primes, UPTO);
            }

            printf("Average time is = %f\n\n", sumExectime / runs);

            sumExectime = 0;
        }
    }
}
```

Γραφικές Παραστάσεις - Χρονομετρήσεις

Για την δοκιμή εναλλακτικών τρόπων διαμοιρασμού της δουλειάς μεταξύ των νημάτων θα χρησιμοποιήσουμε τρεις μεταβλητές με τιμές όπως φαίνεται παρακάτω:

```
numThreads[] = {2, 3, 4}
```

```
omp_sched_t schedules[] = {omp_sched_static, omp_sched_dynamic,  
omp_sched_guided}
```

```
chunkSizes[] = {100000, 250000, 500000, 1000000, 2500000}
```

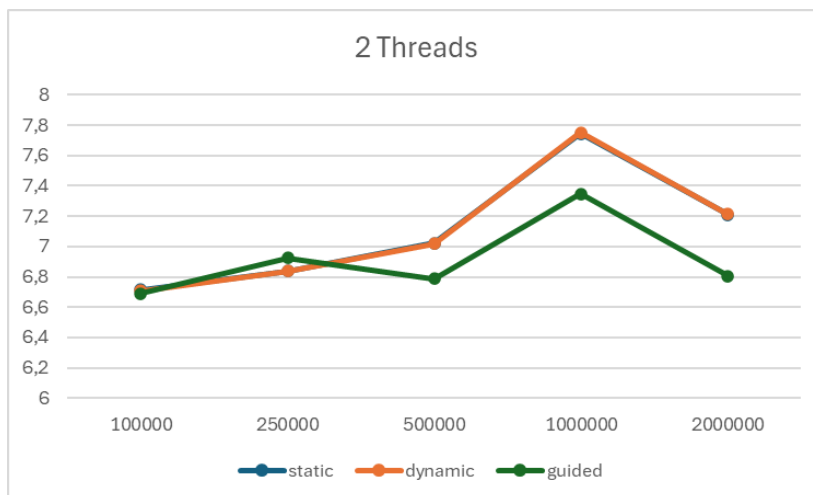
Σημείωση: Κάθε σημείο χρόνου βρέθηκε βάση των μέσω όρο 4 επαναλήψεων.

- Serial 1 Thread:

Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
1	13,068	13,053	13,047	13,066	13,058

- Parallel 2 Threads:

2 threads					
	100000	250000	500000	1000000	2000000
static	6,715	6,838	7,021	7,744	7,211
dynamic	6,708	6,838	7,017	7,753	7,212
guided	6,689	6,925	6,789	7,347	6,807

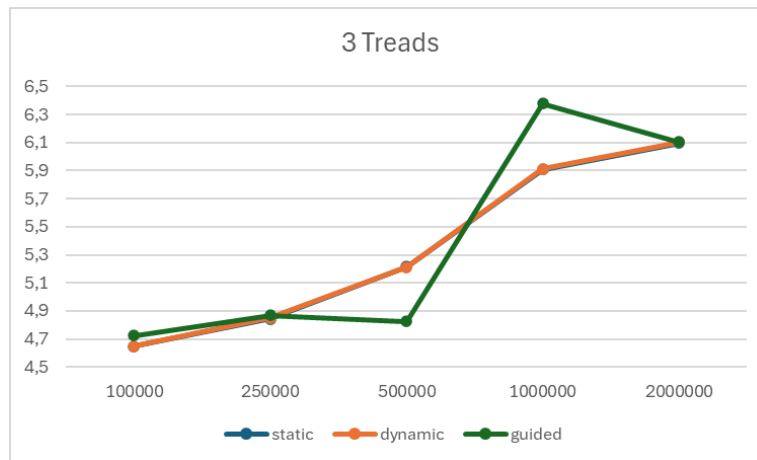


Χρονικό πεδίο: 6,689(min) – 7,753(max)

Ελάχιστος Χρόνος: 6,689 -> guided chunk(100000)

- Parallel 3 Threads:

3threads		100000	250000	500000	1000000	2000000
	static	4,647	4,845	5,214	5,908	6,098
	dynamic	4,649	4,849	5,212	5,911	6,105
	guided	4,725	4,868	4,826	6,378	6,104

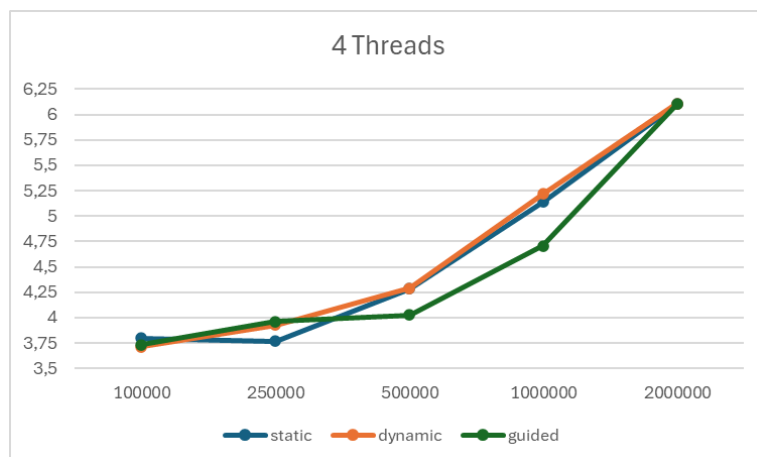


Χρονικό πεδίο: 4,647 (min) – 6,378(max)

Ελάχιστος Χρόνος: 4,647 -> static chunk(100000)

- Parallel 4 Threads:

4threads		100000	250000	500000	1000000	2000000
	static	3,797	3,77	4,287	5,137	6,101
	dynamic	3,716	3,928	4,291	5,218	6,108
	guided	3,734	3,963	4,026	4,707	6,102



Χρονικό πεδίο: 3,716 (min) – 6,108(max)

Ελάχιστος Χρόνος: 3,716 -> dynamic chunk(100000)

Συμπεράσματα

1. Αύξηση του αριθμού των νημάτων βελτιώνει την απόδοση:

Η εκτέλεση με 3 και 4 νήματα παρέχει καλύτερους χρόνους σε σύγκριση με την εκτέλεση με 2 νήματα. Αυτό είναι συνήθως αναμενόμενο σε παράλληλη επεξεργασία, όπου περισσότερα νήματα μπορούν να επεξεργαστούν περισσότερα δεδομένα ταυτόχρονα.

2. Διαχείριση των chunk sizes:

Η επιλογή του μεγέθους του chunk έχει σημαντικό αντίκτυπο στην απόδοση. Παρατηρούμε ότι σε κάθε αριθμό νημάτων αλλά και schedule type μικρότερα chunk δίνουν καλύτερα αποτελέσματα. Αυτό υποδηλώνει ότι η πιο συχνή επανεκχώρηση μικρότερων τμημάτων εργασίας μπορεί να βοηθήσει στην αποφυγή καταστάσεων όπου ορισμένα νήματα ολοκληρώνουν την εργασία τους νωρίτερα και παραμένουν αδρανή.

3. Επιλογή του τύπου διαμοιρασμού (schedule): Τα διάφορα είδη διαμοιρασμού (static, dynamic, guided) δεν δείχνουν μεγάλες διακυμάνσεις στους χρόνους εκτέλεσης. Αυτό το φαινόμενο μπορεί να εξηγηθεί με δύο παρατηρήσεις. Οι επεξεργαστές έχουν ίδιες ταχύτητες, περίπου τον ίδιο φόρτο, και οι επαναλήψεις μεταξύ τους έχουν παρόμοια εργασία να κάνουν.

Άσκηση 2: Θόλωση εικόνων

Το πρόβλημα

Μας δίνεται ένα σειριακό πρόγραμμα το οποίο εφαρμόζει Gaussian blur προκειμένου να θολώσει (ή να ομαλοποιήσει) μία εικόνα. Η συνάρτηση που κάνει τη θόλωση είναι η `gaussian_blur_serial()`. Μας ζητείται να γίνεται η θόλωση παράλληλα χρησιμοποιώντας το OpenMP ως εξής

(χωρίς να αλλάξουμε τον αλγόριθμο, απλά μοιράζοντας σωστά τη δουλειά):

- να συμπληρώσουμε τη συνάρτηση `gaussian_blur_omp_loops()` παραλληλοποιώντας κατάλληλα loop(s).
- να συμπληρώσουμε τη συνάρτηση `gaussian_blur_omp_tasks()` όπου για την παραλληλοποίηση γίνεται χρήση `tasks`.

Ανάλυση Κώδικα `omp_loops`

Διεξήχθησαν δοκιμές παραλληλισμού σε διάφορα επίπεδα του βρόχου επεξεργασίας του Gaussian Blur για να προσδιοριστεί ποιο επίπεδο προσφέρει τη βέλτιστη βελτίωση των επιδόσεων.

Οι μετρήσεις χρόνου εκτέλεσης πραγματοποιήθηκαν σε τέσσερις δοκιμές ανά διαμόρφωση, με χρήση διαφορετικού αριθμού νημάτων.

Στον παραδοτέο κώδικα κρατήσαμε μία από τις βέλτιστες περιπτώσεις.

```
245  omp_set_dynamic(0);
246  //omp_set_nested(1);
247  omp_set_num_threads(4);
248  #pragma omp parallel for private(i, j, col, row, redSum, greenSum, blueSum, weightSum) shared(radius, width, height, imgin, imgout)
249  for (i = 0; i < height; i++)
250  {
```

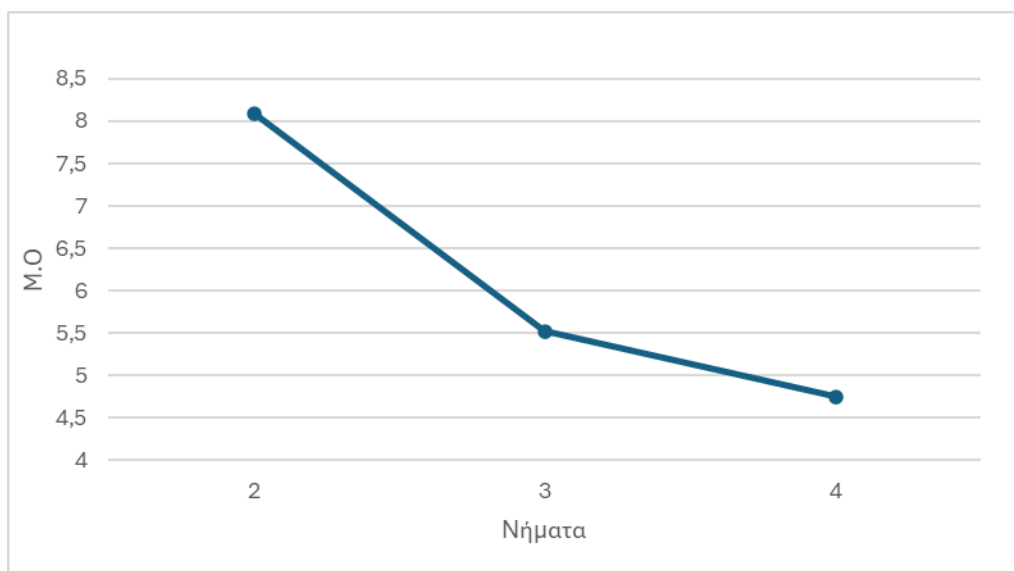
Γραφικές Παραστάσεις - Χρονομετρήσεις

- Σειριακός Κώδικας:

Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
1	16,001	16,023	16,001	15,987	16,003

- Παραλληλισμός μόνο στο 1^ο for:

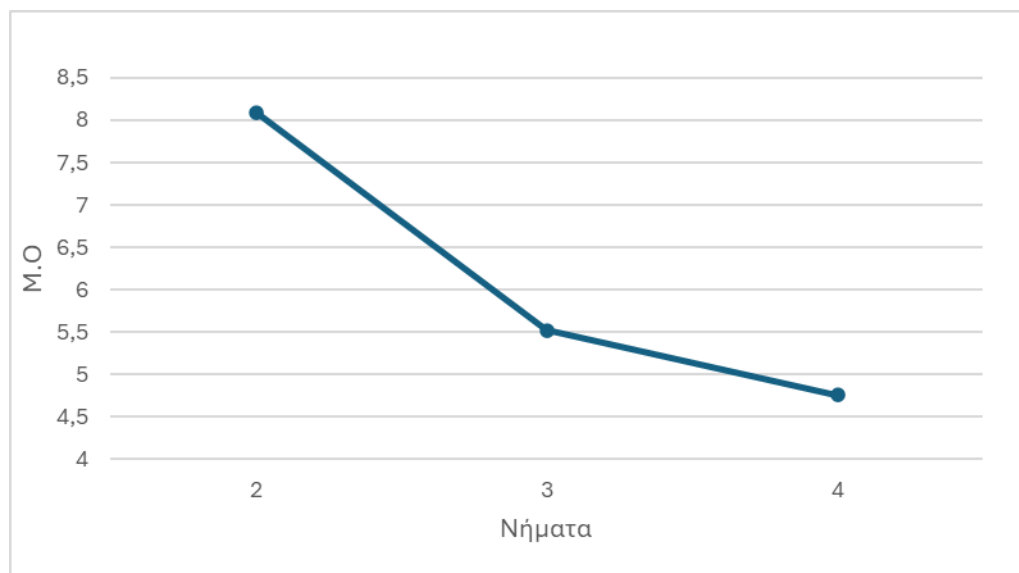
Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
2	8,10	8,087	8,093	8,081	8,09
3	5,513	5,514	5,515	5,554	5,52
4	4,77	4,781	4,789	4,66	4,75



- Η βέλτιστη επίδοση παρατηρήθηκε με 4 νήματα, με μέσο χρόνο 4,75 δευτερόλεπτα. Η χρήση 3 νημάτων έδωσε επίσης καλά αποτελέσματα με μέσο χρόνο 5,52 δευτερόλεπτα.

- Παράλληλισμός μόνο στο 2^ο for:

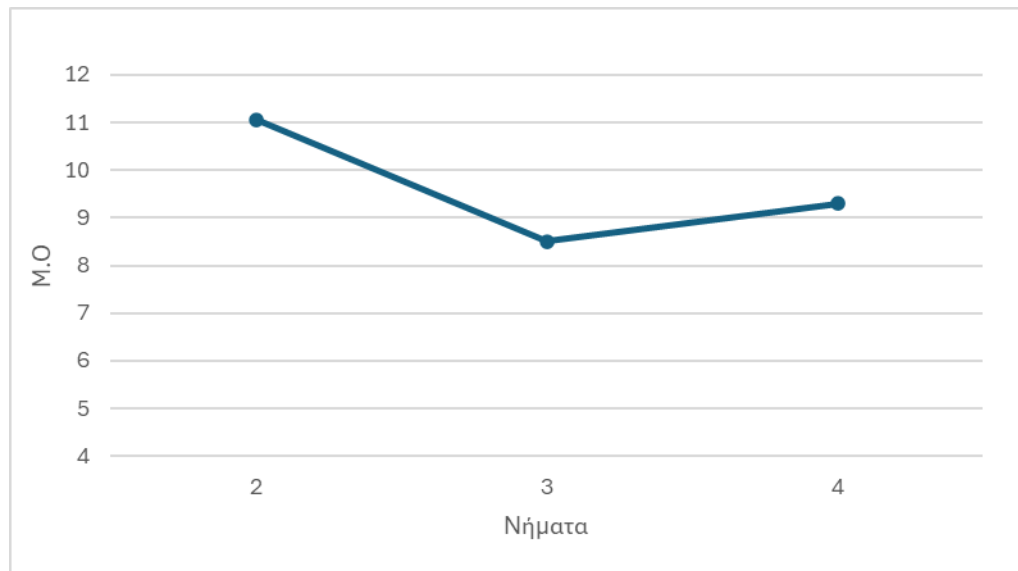
Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
2	8,106	8,10	8,096	8,096	8,09
3	5,527	5,535	5,525	5,529	5,52
4	4,811	4,782	4,799	4,663	4,76



- Παρόμοιες επιδόσεις με το 1^ο for, με τα 4 νήματα να παρέχουν το καλύτερο αποτέλεσμα σε μέσο χρόνο 4,76 δευτερόλεπτα.

- Παραλληλισμός μόνο στο 3^ο for:

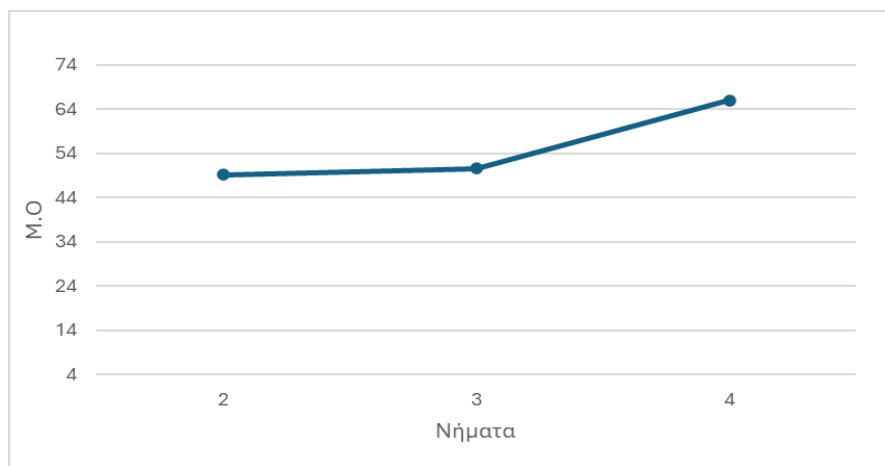
Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
2	11,117	11,042	11,040	11,063	11,06
3	8,58	8,47	8,545	8,431	8,50
4	9,438	9,32	9,254	9,20	9,3



- Μεγαλύτεροι χρόνοι εκτέλεσης σε σχέση με τα προηγούμενα δύο επίπεδα, με τα 3 νήματα να παρέχουν τον καλύτερο μέσο χρόνο στα 8,50 δευτερόλεπτα.

- Παραλληλισμός μόνο στο 4^ο for:

Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
2	49,17	48,071	51,109	48,60	49,23
3	50,25	50,708	50,742	50,810	50,64
4	68,218	65,59	65,68	64,71	66,04



- Σημαντικά υψηλότεροι χρόνοι εκτέλεσης, με τα 2 νήματα να έχουν τον καλύτερο μέσο χρόνο στα 49,23 δευτερόλεπτα.

- Παραλληλισμός στο 1^ο και ταυτόχρονα στο 2^ο :

Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
4 = (2*2)	4,869	4,986	4,811	4,816	4,86

- Ο συνδυασμένος παραλληλισμός των δύο πρώτων βρόχων παρέχει την καλύτερη επίδοση με 4 νήματα (2x2), με μέσο χρόνο 4,86 δευτερόλεπτα.

Συμπεράσματα

Η ανάλυση των δεδομένων δείχνει ότι ο παραλληλισμός των βρόχων επιφέρει σημαντικές διακυμάνσεις στις επιδόσεις. Ο παραλληλισμός του πρώτου ή του δεύτερου βρόχου φαίνεται να είναι η πιο αποτελεσματική επιλογή, με τον παραλληλισμό του τρίτου και του τέταρτου βρόχου να μην είναι τόσο αποδοτικός.

Ο συνδυασμένος παραλληλισμός και των δύο πρώτων βρόχων φαίνεται να παρέχει μια καλή βελτίωση, δείχνοντας το πλεονέκτημα της πολλαπλής επιπέδου παραλληλίας.

Ο παραλληλισμός στα δύο τελευταία for αντί να βελτιώνει επιβραδύνει την εκτέλεση του κώδικα, αυτό το φαινόμενο μπορεί να εξηγηθεί καθώς η επαναλήψεις είναι ελάχιστες, οπότε είναι πιο λογικό να γίνουν σειριακά από ένα νήμα. Ίσως για αρκετά μεγάλο radius να είχε θετικά αποτελέσματα.

Ανάλυση Κώδικα *omp_tasks*

- `#pragma omp single`:
Εξασφαλίζει ότι ο ακόλουθος κώδικας εκτελείται μόνο από ένα νήμα (συνήθως το master νήμα). Αυτό χρησιμοποιείται για τη δημιουργία tasks χωρίς παράλληλο διαμοιρασμό του κώδικα δημιουργίας των tasks.
- `#pragma omp task`:
Καθορίζει ότι ο κώδικας που ακολουθεί πρέπει να εκτελεστεί ως ένα ανεξάρτητο task.

- firstprivate(i, weightSum, redSum, greenSum, blueSum):

Μεταφέρει τις αρχικές τιμές των μεταβλητών i, weightSum, redSum, greenSum, blueSum, εγγυώμενος ότι κάθε task θα έχει το δικό του αντίγραφο αυτών των μεταβλητών.

- private(j, row, col):

Κάθε task διαθέτει τα δικά του αντίγραφα αυτών των μεταβλητών, αποτρέποντας τις συγκρούσεις δεδομένων.

- shared(imgin, imgout, radius, width):

Η χρήση αυτών των μεταβλητών ως κοινόχρηστες (shared) είναι αποδοτικότερη από το να δημιουργηθεί ένα ιδιωτικό αντίγραφο για κάθε task, καθώς οι μεταβλητές απλά διαβάζονται και δεν τροποποιούνται.

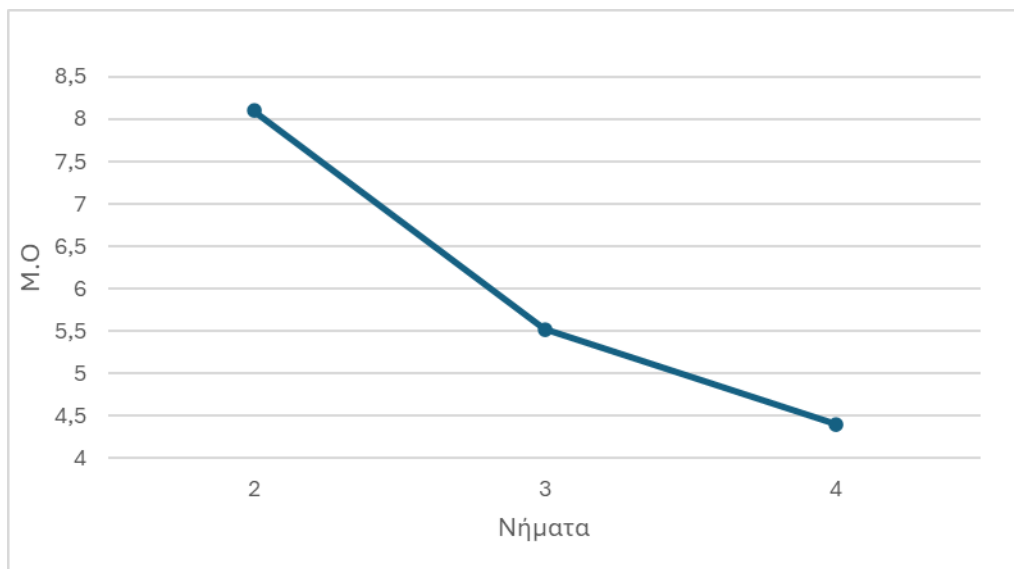
```

295     omp_set_dynamic(0);
296     omp_set_num_threads(4);
297     #pragma omp parallel
298     {
299         #pragma omp single // only the master create the tasks
300         {
301             for (i = 0; i < height; i++)
302             {
303                 #pragma omp task firstprivate(i, weightSum, redSum, greenSum, blueSum) private(j, row, col) shared(imgin, imgout, radius, width)
304                 {
305                     for (j = 0; j < width ; j++)
306                     {

```

Γραφικές Παραστάσεις - Χρονομετρήσεις

Νήματα	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
2	8,107	8,106	8,114	8,111	8,10
3	5,548	5,525	5,521	5,525	5,52
4	4,411	4,398	4,411	4,392	4,40



Συμπεράσματα

Η απόδοση του αλγορίθμου βελτιώνεται καθώς αυξάνεται ο αριθμός των νημάτων. Με 2 νήματα, ο μέσος χρόνος εκτέλεσης είναι περίπου 8,10 δευτερόλεπτα, με 3 νήματα μειώνεται σε 5,52 δευτερόλεπτα, και με 4 νήματα μειώνεται περαιτέρω σε 4,40 δευτερόλεπτα. Αυτό δείχνει μια ξεκάθαρη κλιμάκωση των επιδόσεων σε σχέση με τον αριθμό των νημάτων.

Άσκηση 3: Task dependencies

Οι εξαρτήσεις εργασιών στον OpenMP επιτρέπουν να καθορίζονται συγκεκριμένες σχέσεις μεταξύ των εργασιών, οι οποίες πρέπει να εκτελεστούν σε μια συγκεκριμένη σειρά. Αυτό επιτρέπει στον σχεδιαστή του προγράμματος να προσδιορίζει ποιες εργασίες πρέπει να ολοκληρωθούν πριν αρχίσει η επόμενη εργασία (συγχρονισμός εργασιών), αυξάνοντας την αποδοτικότητα της εκτέλεσης και την κατανόηση του κώδικα.

Πλεονεκτήματα:

1. Βελτιωμένος Παραλληλισμός:

Οι εξαρτήσεις εργασιών διασφαλίζουν ότι οι εργασίες που εξαρτώνται η μία από την άλλη δεν θα εκτελεστούν παράλληλα, αλλά θα τηρούν τη σειρά που απαιτείται για την ορθή εκτέλεση των υπολογισμών. Αυτό επιτρέπει την ασφαλή και αποτελεσματική παράλληλη εκτέλεση χωρίς σφάλματα συγχρονισμού.

2. Αποδοτικότερη Χρήση Πόρων:

Με τον καθορισμό των εξαρτήσεων, οι διαθέσιμοι πόροι (όπως πυρήνες επεξεργαστή) μπορούν να χρησιμοποιηθούν πιο αποτελεσματικά, καθώς οι εργασίες που δεν έχουν εξαρτήσεις μπορούν να εκτελεστούν σε παράλληλο τρόπο.

Ένα μικρό παράδειγμα:

```
void data_flow_example(void){  
    int x, y, z;  
    #pragma omp parallel  
    #pragma omp single{  
        #pragma omp task //task A  
        x = 1;  
        #pragma omp task //task B  
        y = 2;  
        #pragma omp taskwait  
        #pragma omp task //task C  
        z = 3;  
        #pragma omp task //task C  
        printf("x = %d \n", x)  
        #pragma omp task //task D  
        printf("y = %d \n", y)  
    }  
}
```

```
void data_flow_example_with_depends(void){  
    int x, y, z;  
    #pragma omp parallel  
    #pragma omp single{  
        #pragma omp task depend(out:x)//task A  
        x = 1;  
        #pragma omp task depend(out:y)//task B  
        y = 2;  
        #pragma omp task //task C  
        z = 3;  
        #pragma omp task(in: x) //task D  
        printf("x = %d \n", x)  
        #pragma omp task(in: y) //task E  
        printf("y = %d \n", y)  
    }  
}
```

1. Στην υλοποίηση χωρίς task dependencies:

Δεν υπάρχει ευελιξία στην εκτέλεση:

Όλες οι tasks πρέπει να περιμένουν την ολοκλήρωση των προηγούμενων μέσω της `taskwait` πριν προχωρήσουν οποιεσδήποτε άλλες εργασίες, ανεξάρτητα από το αν αυτές εξαρτώνται ή όχι από τα δεδομένα των προηγούμενων tasks.

Πιθανή αδράνεια: Επειδή όλες οι tasks πρέπει να περιμένουν η μία την άλλη, μπορεί να υπάρχουν περίοδοι όπου κάποια νήματα δεν εκτελούν καμία εργασία, αν και θα μπορούσαν να εκτελέσουν tasks που δεν εξαρτώνται από τις ολοκληρωμένες ακόμα tasks.

2. Στην υλοποίηση με task dependencies:

Ευελιξία στην εκτέλεση: Εργασίες που δεν είναι εξαρτημένες μπορούν να εκτελεστούν χωρίς να περιμένουν άλλες (**task C**), αυξάνοντας την αποδοτικότητα του παραλληλισμού.

Καλύτερη αξιοποίηση των πόρων: Κάθε task μπορεί να εκτελεστεί μόλις οι προϋποθέσεις του είναι έτοιμες (π.χ **taskA -> taskD**, **taskB -> taskE**), αποφεύγοντας την αδράνεια και βελτιστοποιώντας τη χρήση των διαθέσιμων νημάτων.